《现代C++编程实现》电子版：https://leanpub.com/c01

C++语言介绍

YouTube频道： hwdong

博客： hwdong-net.github.io

B站: hw-dong

目录

- 为什么要学习C++语言?
- 为什么要学习现代C++?
- 教材说明
- 程序和编程语言
- C++程序的编译与运行

为什么要学习C++?

hwdong
董洪伟

YouTube频道： hwdong

博客： hwdong-net.github.io

C++语言

- C++语言是对C语言的扩展和增强: **面向对象**、**通用算法**（泛型编程)
- 1979年，贝尔实验室Bjarne Stroustrup(C++之父，现在Texas A&M)发明。
- 1998年被ISO委员会批准，2003发布了修正版。2011开始增加了很多新特征：C++11、C++14、C++17、C++20、C++23。

C/C++：**底层控制、性能王者**

C++ 是从 C 语言发展而来的，继承了 C 语言强悍的高性能和底层控制能力

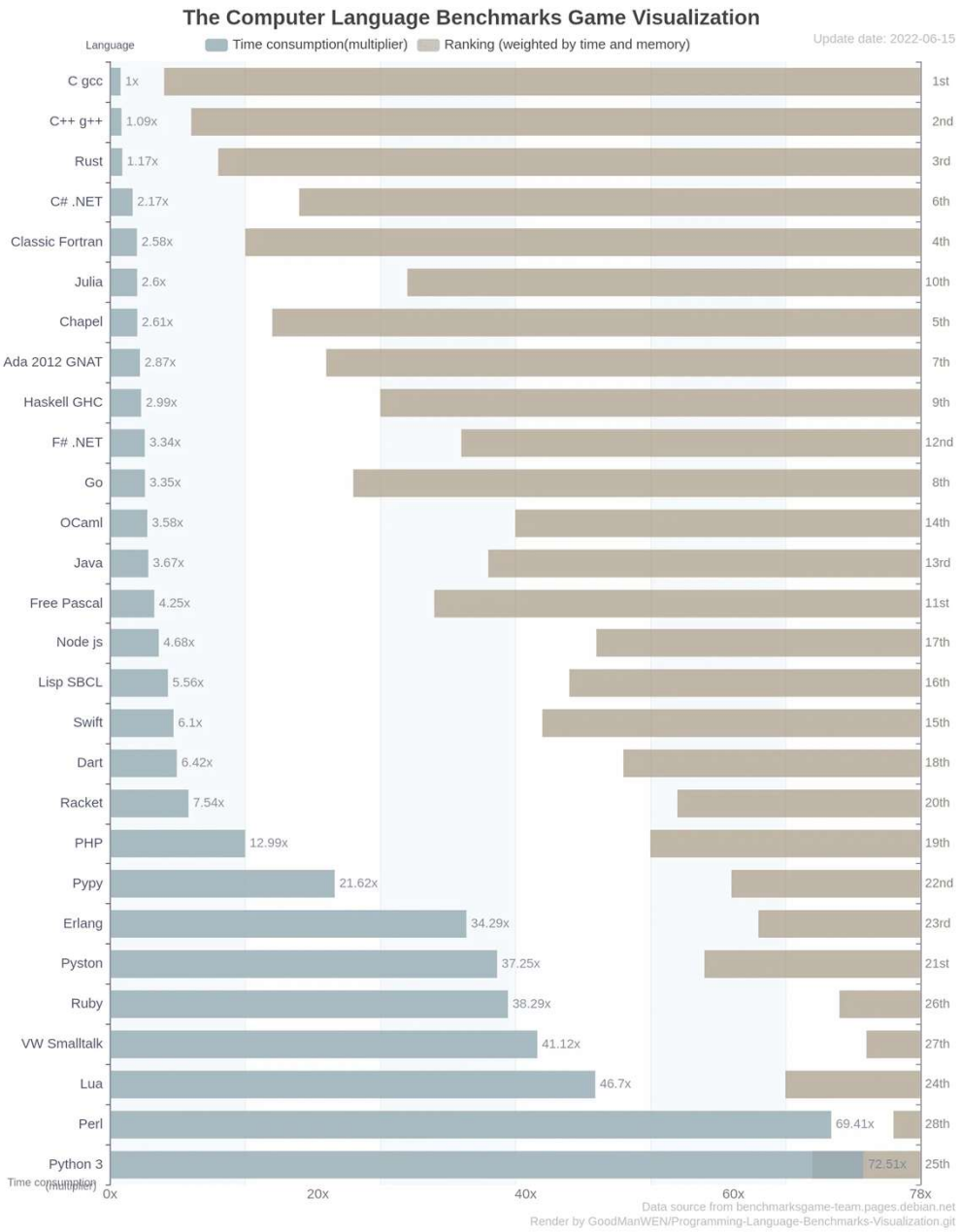
- 高效执行：C++ 代码可以直接编译为高效的机器码，运行速度快，适用于对性能要求极高的场景。
- 手动内存管理：C++ 允许开发者直接管理内存（new/delete、指针操作），可以优化资源使用，提高程序的效率。
- 底层硬件控制：C++ 可以直接访问 CPU 寄存器、内存、IO 端口，非常适合系统编程、驱动开发和嵌入式系统。

- **运行速度对比（C++ vs Java）**：

任务	C++耗时（秒）	Java耗时（秒）
regex-redux	1.83	10.50
n-body	9.42	22.00
binary-trees	3.68	8.39

- **游戏开发性能对比**（FPS）[7]：

框架	C++ (Unreal)	C# (Unity)	Python (Pygame)
平均FPS	60	45	15


The Computer Language Benchmarks Game Visualization
Update date: 2022-06-15
Language
Time consumption(multiplier)
Ranking (weighted by time and memory)
C gcc 1x 1st
C++ g++ 1.09x 2nd
Rust 1.17x 3rd
C# .NET 2.17x 6th
Classic Fortran 2.58x 4th
Julia 2.6x 10th
Chapel 2.61x 5th
Ada 2012 GNAT 2.87x 7th
Haskell GHC 2.99x 9th
F# .NET 3.34x 12nd
Go 3.35x 8th
OCaml 3.58x 14th
Java 3.67x 13rd
Free Pascal 4.25x 11st
Node js 4.68x 17th
Lisp SBCL 5.56x 16th
Swift 6.1x 15th
Dart 6.42x 18th
Racket 7.54x 20th
PHP 12.99x 19th
Pypy 21.62x 22nd
Erlang 34.29x 23rd
Pyston 37.25x 21st
Ruby 38.29x 26th
VW Smalltalk 41.12x 27th
Lua 46.7x 24th
Perl 69.41x 28th
Python 3 72.51x 25th
Time consumption (multiplier)
0x
20x
40x
60x
78x
Data source from benchmarksgame-team.pages.debian.net
Render by GoodManWEN/Programming-Language-Benchmarks-Visualization.git

全能王：从系统软件到应用软件，无所不能

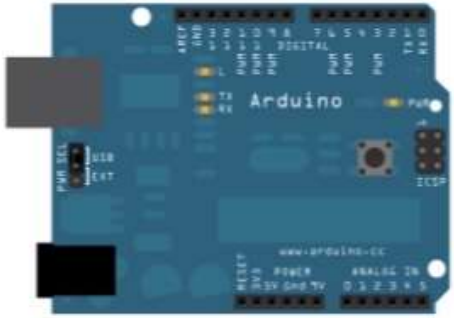
由于其高性能和底层控制能力，C++ 被广泛用于开发各种关键软件：

- 操作系统（如 Windows 部分内核、Linux 组件）。
- 数据库系统（如 MySQL、MongoDB）。
- 网络和浏览器（如 Chrome、Firefox 内核）。
- 图形与多媒体（如 OpenGL、Blender）。
- 游戏和图形引擎（如 Unreal Engine、Unity）。
- 高性能计算（如 TensorFlow、CUDA 计算库）。

- 硬件驱动和操作系统

ARM嵌入式

单片机

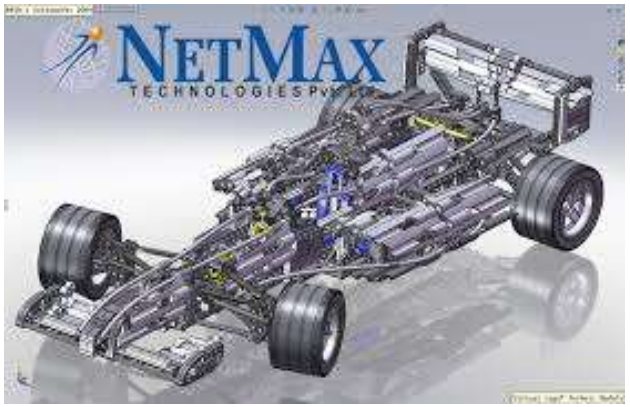


Arduino

- 硬件驱动和操作系统

- 应用软件：游戏、CAD 专业软件

动漫制作



CAD

游戏

- 应用软件：图形计算、高性能计算、人工智能

Tensorlow

Pytorch
Caffe2

NVIDIA CUDA

OpenCL

OpenGL

Microsoft DirectX

- 应用软件：聊天软件

QQ

不可替代的应用场景：嵌入式、实时、高性能

- 在嵌入式系统（如智能手表、工业机器人）、实时系统（如自动驾驶）、图形计算（如OpenCV、Unreal引擎）等领域，C/C++是唯一选择！

培养严谨的编程思维：底层逻辑、高层思想

- 它要求开发者手动管理内存（如new/delete）、理解指针操作、掌握多范式编程（面向对象、泛型、过程式）。
- 这种"知其所以然"的学习过程，能帮助程序员建立对计算机系统架构的深刻认知。
- 许多编程语言（如Java、Python）的底层实现都采用C/C++，学好C++等于掌握了理解其他语言的钥匙。

一句话：提升编程内功

为什么学习现代 C++?

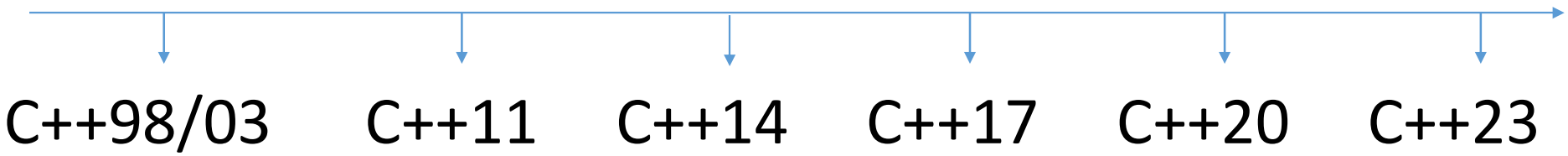
C++11 及更高版本的重要性

```
// 传统 C++
std::vector<int>::iterator it = myVector.begin();

// 现代 C++ (使用 auto)
auto it = myVector.begin();
```

C++ 的演进

- C++ 并非一成不变，而是在不断发展！



- 现代 C++ (C++11 及更高版本) 带来了革命性的改变。
- 掌握现代 C++ 不仅能提升开发效率，还能构建更健壮、更安全、性能更优的应用程序。

更简洁、更易读的代码

- **auto 类型推断**: 简化变量声明，减少冗余。

```
// 传统 C++
std::vector<int>::iterator it = myVector.begin();

// 现代 C++ (auto)
auto it = myVector.begin();
```

更简洁、更易读的代码

- **范围 for 循环**: 简化容器遍历，告别迭代器。

```
// 传统 C++
for (std::vector<int>::iterator it = myVector.begin(); it !=
                                        myVector.end(); ++it) {
    std::cout << *it << std::endl;
}

// 现代 C++ (range-based for loop)
for (int element : myVector) {
    std::cout << element << std::endl;
}
```

更高性能的应用程序

- **移动语义**: 避免不必要的拷贝，大幅提升性能。

  资源转移，而非复制！

- **emplace_back/emplace**: 直接构造对象，避免临时对象。
- **constexpr**: 编译时计算，减少运行时开销。

更安全的内存管理

- **智能指针**: 自动管理内存，告别内存泄漏和悬挂指针。

  unique_ptr: 独占所有权

  shared_ptr: 共享所有权

  weak_ptr: 观察者指针

- **nullptr**: 类型安全的空指针，避免与整数 0 混淆。

拥抱现代化和标准化

- **标准库扩展**: 更多、更强大的标准库组件 (例如 thread, chrono, random, regex)
- 与时俱进: C++ 持续发展，不断引入新特性。
- 行业需求: 越来越多的公司要求使用现代 C++。

提升开发效率

- **代码简化**: 减少代码量，提高开发速度。
- **更强大的工具**: 标准库提供更强大的工具，减少重复造轮子。
- **更好的可维护性**: 简洁的代码更容易维护，降低维护成本。

为什么学习现代 C++?

- 更简洁、更高效、更安全、更易于维护的代码。
- 提高开发效率，降低维护成本。
- 保持与技术发展同步，提高职业竞争力。
- 立即行动，拥抱现代 C++!

教材

《 C++17从入门到精通 》

清华大学出版社 2019年8月

《 现代C++编程实战》

https://leanpub.com/c01 2025

C++17从入门到精通

清华大学出版社 2019年8月

YouTube频道： hwdong

博客： hwdong-net.github.io

写作动机

尽管C++应用广泛，但许多高校的C++课程仍以C++98/03为主，市面教材也常存在这些问题：

- - **讲解冗长**，充斥大量语法细节和历史包袱。
- - **侧重理论**，**缺少**立即可跑的**实战示例**。
- - 默认读者有**C语言或其它语言基础**，对纯零基础不友好。
- - **缺少现代C++特性**，与传统写法频繁对比，容易混淆。
- - 国外所谓“**大师们**”**的书**不适合国内读者

本书特色

1. **直接讲解现代C++标准**：本书围绕现代C++标准进行编写，避免了传统的从C到C++的教学模式，使得读者可以直接学习现代C++的最新特性，避免过时语法的困扰。

2. **简洁明了的语法讲解**：本书强调语法概念的简明扼要和实用性，力求通过浅显易懂的例子帮助读者理解每个语法点，而不是做过于复杂的语法描述。

3. **结合实战案例**：本书精选了游戏编程、数据结构、机器学习、人工智能等多个领域的经典问题，帮助读者将C++的语法知识与实际应用结合，提高编程能力。

4. **适合初学者和有经验的程序员**：无论是编程初学者，还是有一定经验的开发者，本书都能提供相应的帮助，帮助读者逐步掌握C++，并能应对实际的编程挑战。

教学资源

本书资源： https://github.com/hwdong-net/cplusplus17

作者博客： https://hwdong-net.github.io

Youtube频道： https://www.youtube.com/c/hwdong

程序和编程语言

hwdong
董洪伟

YouTube频道： hwdong

博客： hwdong-net.github.io

计算机是什么

- 计算机是一种通用计算设备，能够根据指令处理数据，并产生结果。

计算机的组成

- **输入**：允许计算机从用户接收信息的任何设备。这包括键盘，鼠标，扫描仪和麦克风。
- **处理**：处理信息的计算机组件。中央处理单元（CPU）、图形处理单元(简称GPU)。
- **存储**：存储信息的组件。内存（ROM、RAM）、外存（磁带、磁盘、优盘）
- **输出**：用于向用户显示信息的任何设备。这包括显示器，扬声器和打印机。

CPU
intel
Core™ i7
内存条
msi
Z97 PC Mate

中央处理单元（CPU）

是计算机的大脑，负责计算、处理数据、控制其他设备等

- 算术/逻辑单元（ALU）执行算术和比较操作。
- 控制单元确定下一个要执行的指令。
- 寄存器形成一个高速存储区以保存临时结果。

远程服务器

自动售票机

存储

处理

输入

输出

自动售票机

- 你投币、按按钮选择目的地（**输入**）。
- 机器验证票务、检查余额、扣款、更新库存（**CPU处理**）。
- 它记住票价、剩余票数等信息（**存储**）。
- 打印车票、屏幕显示“谢谢惠顾”（**输出**）。

计算机指令

- 计算机通过执行一系列指令来完成任务，并通常产生某种输出。这和我们日常生活中的做事方式很像。

  例如，“做饭”这个复杂任务可以分解为以下简单步骤：

  1. 取米并放入洗米盆。
  2. 用自来水冲洗大米。
  3. 如果电饭锅未清洗，则清洗电饭锅。
  4. 将米和水放入电饭锅。
  5. 插上电源，按下开关。
  6. 饭煮好后，拔下电源（任务完成）。

计算机指令

- **计算机只能处理最简单的机器指令。**无论多么复杂的任务（计算平均值、渲染3D游戏画面、训练AI模型），**最终都会被分解成CPU能够理解和执行的简单指令**（如加法、减法、比较大小、跳转到另一条指令等）

计算机指令

- **计算机只能处理最简单的机器指令。**无论多么复杂的任务（计算平均值、渲染3D游戏画面、训练AI模型），**最终都会被分解成CPU能够理解和执行的简单指令**（如加法、减法、比较大小、跳转到另一条指令等）

常见的计算机指令类型包括：

- **算术运算**：加、减、乘、除等。
- **逻辑运算**：比较大小、判断相等与否。
- **控制指令**：改变程序执行顺序（如跳转、循环）。

计算机编程

计算机编程的核心是**算法**、**程序**和**编程语言**。

算法是解决问题的步骤和方法；

程序是算法用特定编程语言书写的具体实现；

而编程语言则是人与计算机之间沟通的桥梁，它将人类的逻辑转化为计算机能够执行的指令。

算法

- **算法** 就是完成特定任务或解决问题的一系列明确、有序且有限的步骤。
- 它就像一份详细的菜谱，告诉你要做什么、按什么顺序做，最终得到一道美味的菜肴。

算法

- **制作一杯珍珠奶茶。**我们可以把整个过程分解成以下步骤：

1. 准备材料：红茶包、牛奶、糖、珍珠（事先煮好）、冰块、杯子、吸管。
2. 烧开一壶水。
3. 将红茶包放入杯中，倒入开水，浸泡3-5分钟，取出茶包。
4. 根据口味加入适量糖，搅拌溶解。
5. 加入煮好的珍珠。
6. 倒入牛奶（或奶精），搅拌均匀。
7. 加入冰块，插入吸管。
8. 完成，可以享用了。

1. 准备材料：红茶包、牛奶、糖、珍珠（事先煮好）、冰块、杯子、吸管。
2. 烧开一壶水。
3. 将红茶包放入杯中，倒入开水，浸泡3-5分钟，取出茶包。
4. 根据口味加入适量糖，搅拌溶解。
5. 加入煮好的珍珠。
6. 倒入牛奶（或奶精），搅拌均匀。
7. 加入冰块，插入吸管。
8. 完成，可以享用了。

这8个步骤就是“制作珍珠奶茶”的**算法**。它具有算法的核心特征：

- **明确性**：每一步都清晰无歧义（例如“浸泡3-5分钟”）。
- **顺序性**：步骤必须按特定顺序执行（你不能先加冰块再泡茶）。
- **有限性**：步骤数量有限，最终会结束（得到一杯奶茶）。

算法

“**计算一组数的平均值**”的算法可以描述为：

1. 初始化总和为0，计数器为0。
2. 依次读取每个数，将其加到总和，计数器加1。
3. 当所有数处理完后，用总和除以计数器，得到平均值。
4. 输出平均值。

算法

- 算法是解决问题的**思想**，它不依赖于任何具体的编程语言。你可以用中文、英文、流程图或伪代码来描述它，然后选择一种编程语言（如C++）将其转化为计算机能执行的**程序**。

程序与编程

- **程序**就是让计算机按照你的想法去工作的**指令序列**——计算机必须按顺序一条一条地执行这些指令。它把算法（解决问题的步骤）用计算机能懂的语言写了出来，比如“先做这个，再做那个”。
- **编程**就是写这些指令的过程：你想好步骤，然后用计算机语言写出来，再测试它能不能正常运行，如果出错就修改，直到它能正确工作。

程序与编程

- 我们有了制作珍珠奶茶的算法（步骤1-8）。现在，如果想让一台自动奶茶机按照这个算法工作，就需要把这个算法“翻译”成机器能理解的指令。这个翻译过程就是**编程**，翻译出来的结果就是**程序**。

- 例如，用类似自然语言的伪代码来描述这个程序：

```
1. 准备材料(红茶包，牛奶，糖，珍珠，冰块，杯子，吸管)
2. 烧水()
3. 将红茶包放入杯子()
4. 倒入开水()
5. 等待(3分钟)
6. 取出茶包()
7. 加入糖(适量)
8. 搅拌()
9. 加入珍珠()
10. 倒入牛奶()
11. 搅拌()
12. 加入冰块()
13. 插入吸管()
14. 完成()
```

程序与编程

- 虽然这个伪代码还不是真正的计算机程序，但它已经比算法更接近程序了：它明确了每一步的操作对象和顺序。
- **程序 = 算法 + 编程语言的具体语法。**
- 计算机能直接看懂我们写的程序吗？不能。计算机本质上只认识“开”和“关”两种状态，也就是 1 和 0

二进制

- 计算机底层硬件由大量**晶体管**组成，每个晶体管只有“开”（1）和“关”（0）两种状态。因此，所有数据和指令都用**二进制**表示。
- **一个晶体管可以存储 1 个二进制位（bit，简称 b）**，即 0 或 1
- 多个晶体管可以组合表示更复杂的数值。例如：
  - **8 位（bit）** 组成 **1 字节（Byte，B）**，即 $2^8 = 256$ 种不同的可能值。

二进制

- 多个晶体管可以组合表示更复杂的数值。例如：

- **8 位（bit）** 组成 **1 字节（Byte，B）**，即 $2^8 = 256$ 种不同的可能值。
- **1024 B = 1 KB（千字节，Kilobyte）。**
- **1024 KB = 1 MB（兆字节，Megabyte）。**
- **1024 MB = 1 GB（吉字节，Gigabyte）。**

机器语言

- 计算机内部的指令和数据都以 0 和 1 组成的**二进制代码**形式存储和处理。其中，**机器语言**（Machine Language） 是CPU 能够直接理解和执行的指令集合。
- 每条机器语言指令都是由 0 和 1 组成的二进制码，通常包括**操作码**（opcode，表示做什么操作）和**操作数**（operand，表示操作的对象或数据）。

机器语言

- 用机器语言实现17 + 20的简单操作，对Intel 8086，可能需要几行二进制代码：

```
1011 0000 0001 0001    // 将 17 存入 AL 寄存器
0000 0100 0001 0100    // 将 20 加到 AL 寄存器
1010 0010 0100 1000 0000 0000    // 将 AL 的值存入内存地址 0x0048
```

机器语言

- 用机器语言实现17 + 20的简单操作，对Intel 8086，可能需要几行二进制代码：

```
1011 0000 0001 0001    // 将 17 存入 AL 寄存器
0000 0100 0001 0100    // 将 20 加到 AL 寄存器
1010 0010 0100 1000 0000 0000    // 将 AL 的值存入内存地址 0x0048
```

- `1011 0000` 是 **操作码（Opcode）**，表示“将后面的数存入寄存器 AL”。
- `0001 0001` 是 **立即数（Immediate Value）**，即 17。
- `0000 0100` 指示 AL 执行加法操作，`0001 0100` 是 20 的二进制表示。
- `1010 ...` 负责将结果存入指定内存地址 `0x0048`（采用 **小端存储**）。

机器语言

- 用机器语言进行编程，甚至通过穿孔卡片输入指令。这种方式既繁琐又容易出错，极大地限制了计算机的使用效率。

汇编语言（Assembly Language）

- 为了简化机器语言的编写，引入了 **汇编语言（Assembly Language）**。汇编语言使用 **助记符（Mnemonic）** 代替二进制代码，使程序更易读，例如：

```
MOV AL, 17D          ; 将 17 存入 AL 寄存器
ADD AL, 20D          ; 将 20 加到 AL 寄存器
MOV [0048H], AL      ; 将 AL 的值存入内存地址 0x0048
```

汇编语言（Assembly Language）的特点

- 每条汇编指令 **对应一条机器指令。**
- 人类可读，但仍然需要较高的技术背景。
- 需要 **汇编程序（Assembler）** 将汇编代码转换为机器语言，计算机才能执行。

```
MOV AL, 17D          ; 将 17 存入 AL 寄存器
ADD AL, 20D          ; 将 20 加到 AL 寄存器
MOV [0048H], AL      ; 将 AL 的值存入内存地址 0x0048
```

高级语言（High-Level Languages）

- 人们开发出了高级语言，比如 C、C++、Python、Java 等。
- 在高级语言里，一行代码通常能对应好几条甚至几十条机器指令，而且写法更接近我们日常说话的逻辑。

```
sum = 17 + 20;
```

高级语言（High-Level Languages）的特点

- **可读性更强**，类似人类语言，易于理解和学习。
- **编程效率高**，减少底层细节，减少重复性操作，使开发者可以专注于算法和逻辑。
- **可移植性好**，可以在不同的计算机和操作系统上运行（重新编译即可），而不需要针对特定 CPU 进行修改。

```
sum = 17 + 20;
```

机器语言、汇编语言、高级语言对比

语言类型	示例	特点
机器语言	`1011 0000 0001 0001`	计算机直接执行，效率最高，但极难理解
汇编语言	`MOV AL, 17D`	可读性稍强，但仍复杂、依赖CPU
高级语言	`sum = 17 + 20;`	可读性好、跨平台、易编写

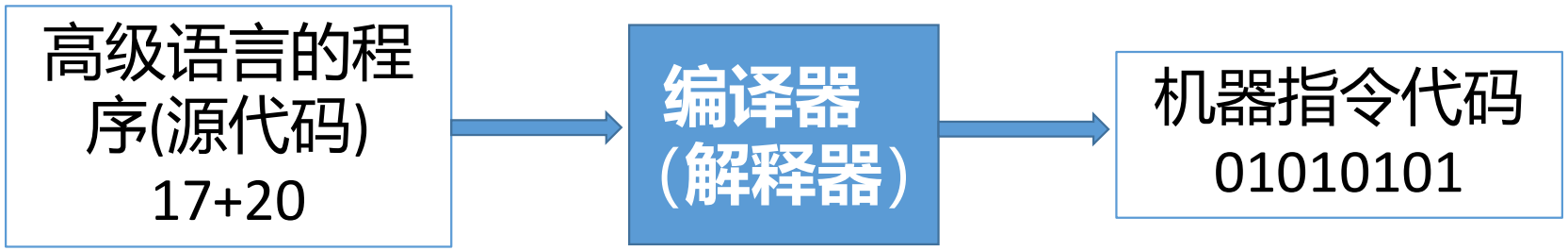
编译器、解释器

- 高级语言编写的程序在计算机执行之前，必须转换为机器语言。这种转换方式主要有两种：**编译（Compilation）** 和**解释（Interpretation）**。
- **编译型语言（Compiled Language）**：程序源代码被整体翻译为可执行的二进制文件，然后执行。运行快，适合高性能场景。
- **解释型语言（Interpreted Language）**：程序源代码被逐行翻译并执行。方便调试，但通常慢。

编译器、解释器

- **编译器或解释器**：将高级语言编程的程序转化为机器指令程序

sum = 17 + 20



编译器与解释器的区别

对比项	编译器（Compiler）	解释器（Interpreter）
工作方式	先将整个程序翻译成机器代码，然后执行	逐行翻译并执行
执行效率	运行速度快，因为已转换为机器语言	运行速度较慢，因为需要边翻译边执行
错误检测	只有在编译时发现错误	可以在执行时逐行发现错误
常见语言	C、C++、Java（编译为字节码后解释执行）	Python、JavaScript、Shell 脚本

编译型语言的优势

- 编译器能对代码进行全局优化，提高运行效率。
- 因此，**C++ 在高性能计算、系统编程、嵌入式开发等领域具有广泛应用**。相比解释型语言，编译后的 C++ 程序通常运行更快，并且可以直接操纵硬件。

C++ 语言概述

- C++ 由 **Bjarne Stroustrup** 于 **1979-1980 年** 在 **贝尔实验室（Bell Labs）** 开发，初衷是对 **C 语言** 进行扩展，使其支持 **面向对象编程（OOP）**。

- C++ 继承了 C 语言的 **高效性** 和 **底层控制能力**，同时增加了 **面向对象特性**（如类、继承、多态等）。
- 由于其强大的性能和灵活性，C++ 可运行在 **Windows**、**macOS**、**Linux**、**UNIX** 等多种操作系统上。
- 经过多年的发展，C++ 一直是最受欢迎的编程语言之一，长期位居 **全球编程语言排行榜前五**。

C++ 的应用领域

- **操作系统与系统软件**（Windows、Linux、macOS 内核，驱动程序）
- **嵌入式开发**（物联网、汽车电子、智能家居、芯片编程）
- **游戏开发与游戏引擎**（Unity、Unreal Engine、CryEngine）
- **高性能计算**（人工智能、数值计算、科学模拟）
- **金融系统**（银行、证券、高频交易）
- **数据库与服务器**（MySQL、PostgreSQL、Web 服务器）
- **计算机图形学**（图像处理、3D 建模、CAD/CAM）

现代 C++ 的演进

- **C++98（1998 年）**：最早的 C++ 标准，基于 C 语言，增加了面向对象特性。
- **C++11（2011 年）**：现代 C++ 的起点，新增 **Lambda 表达式、auto 关键字、智能指针、并发库** 等特性。
- **C++14（2014 年）**：对 C++11 进行改进，增加了更简洁的语法和库优化。
- **C++17（2017 年）**：引入 **结构化绑定（Structured Binding）**、std::optional、**并行算法** 等特性。
- **C++20（2020 年）**：大幅更新，增加 **模块化（Modules）**、**协程（Coroutines）**、**概念（Concepts）** 等新功能。

为何直接学习现代 C++?

- 高校教学中，许多课程仍然停留在 **C 语言或 C++98 时代**，导致学生学习的知识 **落后于工业界。**

- **减少学习负担**，避免学习过时的 C 风格代码。
- **掌握更现代、高效的编程方法**（如 `auto` 关键字、智能指针、Lambda 等）。
- **与工业界无缝对接**，更快适应企业开发需求。

C++ 程序开发步骤

1. **理解问题**：明确程序要做什么，输入是什么，输出是什么。
2. **设计算法**：想出解决问题的具体步骤。
3. **编写程序**：用C++语言把步骤写出来。
4. **测试程序**：运行程序，检查结果是否正确，如果有错就修改。

举例：计算两个整数的和

假设要编写一个程序，让用户输入两个整数，然后输出它们的和。

1. **理解问题**
    - 输入：用户从键盘输入的两个整数
    - 输出：这两个数的和，显示在屏幕上

2. **设计算法**

- 让用户输入第一个数，存起来
- 让用户输入第二个数，存起来
- 计算两个数的和
- 输出结果

3. 编写程序

```cpp
#include <iostream>

int main() {
    int a, b;           // 定义两个变量，用来存用户输入的数

    std::cout << "请输入第一个整数：";
    std::cin >> a;      // 读取第一个数

    std::cout << "请输入第二个整数：";
    std::cin >> b;      // 读取第二个数

    int sum = a + b;    // 计算和

    std::cout << "两个数的和是：" << sum << std::endl;
    return 0;
}
```

3.编译运行程序

在某个“C++ online Compiler”网站上运行这个程序

https://www.onlinegdb.com/online_c++_compiler

4. **测试程序**

- 运行程序，输入 5 和 3，看看是否输出 8
- 输入 -2 和 7，看看是否输出 5
- 如果结果不对，检查代码哪里写错了

关注我

博客： hwdong-net.github.io

Youtube频道

hwdong

@hwdong · 5.01K subscribers · 558 videos

博客：https://hwdong-net.github.io >

youtube.com/c/4kRealSound and 4 more links